Name-Somesh Rathore_EnrollmentNo.-GGV/24/05167

```java
//1.Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

import java.io.*;
import java.util.*;

class Farmer {
    String name;
    String fruit;
    String city;
    int quantity;

    Farmer(String name, String fruit, String city, int quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity;
    }
}

public class FarmerData {

    // Function to write data to Notepad file
    public static void writeToFile(List<Farmer> farmers, String filename) {
        try {
            BufferedWriter writer = new BufferedWriter(new FileWriter(filename));

            writer.write(String.format("%-15s %-15s %-15s %-10s\n", "Name", "Fruit", "City", "Quantity"));
            writer.write("------------------------------------------------------------\n");

            for (Farmer f : farmers) {
                writer.write(String.format("%-15s %-15s %-15s %-10d\n",
                        f.name, f.fruit, f.city, f.quantity));
            }

            writer.close();
            System.out.println("Data saved to " + filename);

        } catch (IOException e) {
            System.out.println("Error writing file: " + e.getMessage());
        }
    }

    // Search function
    public static void searchFarmer(List<Farmer> farmers, String searchName) {
        boolean found = false;
```

```java

        for (Farmer f : farmers) {
            if (f.name.equalsIgnoreCase(searchName)) {
                System.out.println("\nFarmer Found:");
                System.out.println("Name: " + f.name);
                System.out.println("Fruit: " + f.fruit);
                System.out.println("City: " + f.city);
                System.out.println("Quantity: " + f.quantity);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Farmer not found!");
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        List<Farmer> farmers = new ArrayList<>();

        // Input for 10 farmers
        for (int i = 0; i < 10; i++) {
            System.out.println("\nEnter details for Farmer " + (i + 1));

            System.out.print("Name: ");
            String name = sc.nextLine();

            System.out.print("Fruit: ");
            String fruit = sc.nextLine();

            System.out.print("City: ");
            String city = sc.nextLine();

            System.out.print("Quantity: ");
            int quantity = sc.nextInt();
            sc.nextLine(); // consume newline

            farmers.add(new Farmer(name, fruit, city, quantity));
        }

        // Save before sorting
        writeToFile(farmers, "before_sort.txt");

        // Sort by quantity
        Collections.sort(farmers, new Comparator<Farmer>() {
            public int compare(Farmer f1, Farmer f2) {
                return Integer.compare(f1.quantity, f2.quantity);
            }
        });

        // Save after sorting
```

```
        writeToFile(farmers, "after_sort.txt");

        // Search operation
        System.out.print("\nEnter farmer name to search: ");
        String searchName = sc.nextLine();

        searchFarmer(farmers, searchName);

        sc.close();
    }
}

//OUTPUT:

Enter details for Farmer 1
Name: Ravi
Fruit: apple
City: Bhopal
Quantity: 45

Enter details for Farmer 2
Name: Amit
Fruit: orange
City: Indore
Quantity: 74

Enter details for Farmer 3
Name: Sohan
Fruit: apple
City: Raipur
Quantity: 45

Enter details for Farmer 4
Name: Neha
Fruit: lichi
City: Jaipur
Quantity: 52

Enter details for Farmer 5
Name: Karan
Fruit: Delhi
City: orange
Quantity: 35

Enter details for Farmer 6
Name: Pooja
Fruit: mango
City: luchnow
Quantity: 65

Enter details for Farmer 7
Name: Rahul
Fruit: banana
City: Kanpur
```

```
Quantity: 70

Enter details for Farmer 8
Name: Simran
Fruit: lichi
City: Patna
Quantity: 56

Enter details for Farmer 9
Name: Arjun
Fruit: apple
City: Pune
Quantity: 84

Enter details for Farmer 10
Name: Rohit
Fruit: banana
City: Surat
Quantity: 85
Data saved to before_sort.txt
Data saved to after_sort.txt

Enter farmer name to search: Pooja

Farmer Found:
Name: Pooja
Fruit: mango
City: luchnow
Quantity: 65

//2.Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

import java.util.*;

class Fruit {
    String name;
    int quantity;
    String quality;

    Fruit(String name, int quantity, String quality) {
        this.name = name;
        this.quantity = quantity;
        this.quality = quality.toLowerCase();

        // Apply condition: if quantity > 1000 → store half
        if (this.quantity > 1000) {
            this.quantity = this.quantity / 2;
        }
    }
}

public class FruitQuality {
```

```java

    // Search function
    public static void searchFruit(List<Fruit> fruits, String name, String quality) {
        boolean found = false;

        for (Fruit f : fruits) {
            if (f.name.equalsIgnoreCase(name) && f.quality.equalsIgnoreCase(quality)) {
                System.out.println("\nFruit Found:");
                System.out.println("Name: " + f.name);
                System.out.println("Quantity: " + f.quantity);
                System.out.println("Quality: " + f.quality);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("\nNo matching fruit found!");
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        List<Fruit> fruits = new ArrayList<>();

        String[] fruitNames = {"Mango", "Apple", "Papaya"};

        // Input
        for (String fname : fruitNames) {
            System.out.println("\nEnter details for " + fname);

            System.out.print("Quantity: ");
            int quantity = sc.nextInt();
            sc.nextLine(); // consume newline

            System.out.print("Quality (good / average / bad): ");
            String quality = sc.nextLine();

            fruits.add(new Fruit(fname, quantity, quality));
        }

        // Search
        System.out.println("\n--- Search Fruit ---");
        System.out.print("Enter fruit name: ");
        String searchName = sc.nextLine();

        System.out.print("Enter quality: ");
        String searchQuality = sc.nextLine();

        searchFruit(fruits, searchName, searchQuality);

        sc.close();
```

```
    }
}

//OUTPUT:

Enter details for Mango
Quantity: 1500
Quality (good / average / bad): good

Enter details for Apple
Quantity: 500
Quality (good / average / bad): average

Enter details for Papaya
Quantity: 800
Quality (good / average / bad): good

--- Search Fruit ---
Enter fruit name: mango
Enter quality: good

Fruit Found:
Name: Mango
Quantity: 750
Quality: good

//3.There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.
The available orchards in a tree-view.
Search for a given fruit in the available orchards after taking input from the user & print the tree.

import java.util.*;

// Node class for tree
class Node {
    String orchardName;
    String fruit;
    String quality;
    List<Node> children;

    Node(String orchardName, String fruit, String quality) {
        this.orchardName = orchardName;
        this.fruit = fruit;
        this.quality = quality;
        this.children = new ArrayList<>();
    }
}

public class OrchardTree {

    // Function to print tree
    public static void printTree(Node root, String searchFruit) {
```

```java
        for (Node child : root.children) {

            // Highlight if matches search
            if (child.fruit.equalsIgnoreCase(searchFruit)) {
                System.out.println("➡ " + child.orchardName + " -> "
                    + child.fruit + " (" + child.quality + ") [FOUND]");
            } else {
                System.out.println("   " + child.orchardName + " -> "
                    + child.fruit + " (" + child.quality + ")");
            }
        }
    }

    // Search function
    public static boolean searchFruit(Node root, String fruit) {
        boolean found = false;

        for (Node child : root.children) {
            if (child.fruit.equalsIgnoreCase(fruit)) {
                found = true;
            }
        }
        return found;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Root node (Tree)
        Node root = new Node("Orchards", "", "");

        // Adding orchards
        root.children.add(new Node("Orchard A", "Apple", "Good"));
        root.children.add(new Node("Orchard B", "Litchi", "Good"));
        root.children.add(new Node("Orchard C", "Mango", "Good"));
        root.children.add(new Node("Orchard D", "Apple", "Bad"));

        // Display initial tree
        System.out.println("Tree View of Orchards:");
        printTree(root, ""); // no highlight

        // User input
        System.out.print("\nEnter fruit to search: ");
        String searchFruit = sc.nextLine();

        // Search result
        if (searchFruit(root, searchFruit)) {
            System.out.println("\nFruit found in the orchards!");
        } else {
            System.out.println("\nFruit NOT found!");
        }

        // Print tree with highlight
```

```
        System.out.println("\nUpdated Tree View:");
        printTree(root, searchFruit);

        sc.close();
    }
}

//OUTPUT:

Tree View of Orchards:
   Orchard A -> Apple (Good)
   Orchard B -> Litchi (Good)
   Orchard C -> Mango (Good)
   Orchard D -> Apple (Bad)

Enter fruit to search: litchi

Fruit found in the orchards!

Updated Tree View:
   Orchard A -> Apple (Good)
? Orchard B -> Litchi (Good) [FOUND]
   Orchard C -> Mango (Good)
   Orchard D -> Apple (Bad)

//4.Take a map provided as a graph & display all the subtrees of connected points.

import java.util.*;

public class GraphSubtrees {

    // DFS function
    public static void dfs(int node, Map<Integer, List<Integer>> graph,
                           Set<Integer> visited, List<Integer> component) {

        visited.add(node);
        component.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited.contains(neighbor)) {
                dfs(neighbor, graph, visited, component);
            }
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Input number of nodes and edges
        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
```

```java
        int e = sc.nextInt();

        // Create graph
        Map<Integer, List<Integer>> graph = new HashMap<>();

        for (int i = 1; i <= n; i++) {
            graph.put(i, new ArrayList<>());
        }

        // Input edges
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u); // undirected graph
        }

        // Visited set
        Set<Integer> visited = new HashSet<>();

        // Find connected components (subtrees)
        System.out.println("\nSubtrees (Connected Components):");

        for (int i = 1; i <= n; i++) {
            if (!visited.contains(i)) {

                List<Integer> component = new ArrayList<>();
                dfs(i, graph, visited, component);

                System.out.println(component);
            }
        }

        sc.close();
    }
}

//OUTPUT:

Enter number of nodes: 7
Enter number of edges: 5
Enter edges (u v):
4 5
4 2
1 2
3 4
3 5

Subtrees (Connected Components):
[1, 2, 4, 5, 3]
[6]
[7]
```

```java

//5.Enter entry point & goal point & produce the matching subtree of the graph.

import java.util.*;

public class GraphPath {

    // DFS to find path
    public static boolean findPath(int current, int goal,
                                   Map<Integer, List<Integer>> graph,
                                   Set<Integer> visited,
                                   List<Integer> path) {

        visited.add(current);
        path.add(current);

        // If goal found
        if (current == goal) {
            return true;
        }

        for (int neighbor : graph.get(current)) {
            if (!visited.contains(neighbor)) {
                if (findPath(neighbor, goal, graph, visited, path)) {
                    return true;
                }
            }
        }

        // Backtrack if not found
        path.remove(path.size() - 1);
        return false;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Input graph
        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        Map<Integer, List<Integer>> graph = new HashMap<>();

        for (int i = 1; i <= n; i++) {
            graph.put(i, new ArrayList<>());
        }

        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
```

```
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u);
        }

        // Input entry & goal
        System.out.print("\nEnter Entry Point: ");
        int start = sc.nextInt();

        System.out.print("Enter Goal Point: ");
        int goal = sc.nextInt();

        // Find path
        Set<Integer> visited = new HashSet<>();
        List<Integer> path = new ArrayList<>();

        boolean found = findPath(start, goal, graph, visited, path);

        if (found) {
            System.out.println("\nMatching Subtree (Path): " + path);
        } else {
            System.out.println("\nNo path exists between given points!");
        }

        sc.close();
    }
}

//OUTPUT:

Enter number of nodes: 7
Enter number of edges: 6
Enter edges (u v):
4 5
5 6
2 5
3 4
4 1
1 3

Enter Entry Point: 1
Enter Goal Point: 6

Matching Subtree (Path): [1, 4, 5, 6]


//6.Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

import java.util.*;

class Pair {
    int node, weight;
```

```java

    Pair(int node, int weight) {
        this.node = node;
        this.weight = weight;
    }
}

public class DijkstraShortestPath {

    public static void dijkstra(Map<Integer, List<Pair>> graph,
                                int start, int end, int n) {

        // Distance array
        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);

        // Parent array (for path)
        int[] parent = new int[n + 1];
        for (int i = 0; i <= n; i++) parent[i] = i;

        // Min heap
        PriorityQueue<Pair> pq = new PriorityQueue<>(
            Comparator.comparingInt(a -> a.weight)
        );

        dist[start] = 0;
        pq.add(new Pair(start, 0));

        while (!pq.isEmpty()) {
            Pair current = pq.poll();

            for (Pair neighbor : graph.get(current.node)) {

                int newDist = dist[current.node] + neighbor.weight;

                if (newDist < dist[neighbor.node]) {
                    dist[neighbor.node] = newDist;
                    pq.add(new Pair(neighbor.node, newDist));
                    parent[neighbor.node] = current.node;
                }
            }
        }

        // If no path
        if (dist[end] == Integer.MAX_VALUE) {
            System.out.println("No path exists!");
            return;
        }

        // Reconstruct path
        List<Integer> path = new ArrayList<>();
        int current = end;

        while (parent[current] != current) {
```

```java
            path.add(current);
            current = parent[current];
        }
        path.add(start);
        Collections.reverse(path);

        // Output
        System.out.println("\nShortest Path: " + path);
        System.out.println("Total Cost: " + dist[end]);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        Map<Integer, List<Pair>> graph = new HashMap<>();

        for (int i = 1; i <= n; i++) {
            graph.put(i, new ArrayList<>());
        }

        System.out.println("Enter edges (u v weight):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            int w = sc.nextInt();

            graph.get(u).add(new Pair(v, w));
            graph.get(v).add(new Pair(u, w)); // undirected
        }

        System.out.print("\nEnter start node: ");
        int start = sc.nextInt();

        System.out.print("Enter end node: ");
        int end = sc.nextInt();

        dijkstra(graph, start, end, n);

        sc.close();
    }
}

//OUTPUT:

Enter number of nodes: 9
Enter number of edges: 8
Enter edges (u v weight):
```

```
4 5 8
1 2 5
3 4 6
6 4 2
7 6 5
3 5 7
4 7 4
1 5 8

Enter start node: 1
Enter end node: 7

Shortest Path: [1, 5, 4, 7]
Total Cost: 20

//7.Write a program to demonstrate the use of all keywords made for exception handling.

import java.util.*;

class ExceptionDemo {

    // Method using 'throws'
    public static void checkAge(int age) throws Exception {
        if (age < 18) {
            // using 'throw'
            throw new Exception("Age must be 18 or above!");
        } else {
            System.out.println("Age is valid.");
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            // TRY block
            System.out.print("Enter a number: ");
            int num = sc.nextInt();

            int result = 100 / num; // may cause ArithmeticException
            System.out.println("Result: " + result);

            System.out.print("\nEnter your age: ");
            int age = sc.nextInt();

            // calling method with 'throws'
            checkAge(age);

        }
        catch (ArithmeticException e) {
            // specific catch
            System.out.println("Error: Cannot divide by zero!");
```

```java
        }
        catch (InputMismatchException e) {
            System.out.println("Error: Invalid input!");
        }
        catch (Exception e) {
            // general catch
            System.out.println("Exception: " + e.getMessage());
        }
        finally {
            // always executes
            System.out.println("\nFinally block executed.");
            sc.close();
        }
    }
}

//OUTPUT:

Enter a number: 6
Result: 16

Enter your age: 22
Age is valid.

Finally block executed.

//8.Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

import java.util.*;

// Custom Exception
class NoMoneyException extends Exception {
    public NoMoneyException(String message) {
        super(message);
    }
}

class BankAccount {
    double balance;

    BankAccount(double balance) {
        this.balance = balance;
    }

    // Withdraw method
    public void withdraw(double amount) throws NoMoneyException {

        if (amount > balance) {
            throw new NoMoneyException("Insufficient balance! Cannot withdraw " + amount);
        } else {
            balance -= amount;
            System.out.println("Withdrawal successful!");
```

```
            System.out.println("Remaining Balance: " + balance);
        }
    }
}

public class BankAccountDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter initial balance: ");
        double balance = sc.nextDouble();

        BankAccount account = new BankAccount(balance);

        System.out.print("Enter amount to withdraw: ");
        double amount = sc.nextDouble();

        try {
            account.withdraw(amount);
        }
        catch (NoMoneyException e) {
            System.out.println("Exception: " + e.getMessage());
        }
        finally {
            System.out.println("Transaction completed.");
            sc.close();
        }
    }
}

//OUTPUT:

Enter initial balance: 50000
Enter amount to withdraw: 35000
Withdrawal successful!
Remaining Balance: 15000.0
Transaction completed.

//9.Write a program which enables to use a platform by a train one at a time.

class Platform {

    // synchronized method → only one train allowed
    public synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is waiting for platform...");

        try {
            System.out.println(trainName + " is using the platform.");
            Thread.sleep(2000); // simulate time on platform
            System.out.println(trainName + " has left the platform.\n");
        } catch (InterruptedException e) {
            System.out.println(e);
```

```
        }
    }
}

// Train class (Thread)
class Train extends Thread {
    Platform platform;
    String trainName;

    Train(Platform platform, String trainName) {
        this.platform = platform;
        this.trainName = trainName;
    }

    public void run() {
        platform.usePlatform(trainName);
    }
}

public class TrainPlatform {
    public static void main(String[] args) {

        Platform platform = new Platform();

        // Multiple trains
        Train t1 = new Train(platform, "Train A");
        Train t2 = new Train(platform, "Train B");
        Train t3 = new Train(platform, "Train C");

        // Start threads
        t1.start();
        t2.start();
        t3.start();
    }
}

//OUTPUT:

Train A is waiting for platform...
Train A is using the platform.
Train A has left the platform.

Train B is waiting for platform...
Train B is using the platform.
Train B has left the platform.

Train C is waiting for platform...
Train C is using the platform.
Train C has left the platform.


//10.Write a programme of Restaurant and Waiter, which accept order if kitchen if free
and sheff deliver orders if he is free.

```

```java

class Kitchen {

    private boolean isOrderReady = false;
    private String order;

    // Waiter places order
    public synchronized void placeOrder(String order) {

        // Wait if previous order not yet served
        while (isOrderReady) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }

        this.order = order;
        System.out.println("Waiter placed order: " + order);

        isOrderReady = true;
        notify(); // notify chef
    }

    // Chef prepares order
    public synchronized void prepareOrder() {

        // Wait if no order available
        while (!isOrderReady) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }

        System.out.println("Chef is preparing: " + order);

        try {
            Thread.sleep(2000); // simulate cooking time
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("Chef delivered: " + order + "\n");

        isOrderReady = false;
        notify(); // notify waiter
    }
}

// Waiter thread
class Waiter extends Thread {
```

```
    Kitchen kitchen;

    Waiter(Kitchen kitchen) {
        this.kitchen = kitchen;
    }

    public void run() {
        String[] orders = {"Pizza", "Burger", "Pasta"};

        for (String order : orders) {
            kitchen.placeOrder(order);

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {}
        }
    }
}

// Chef thread
class Chef extends Thread {
    Kitchen kitchen;

    Chef(Kitchen kitchen) {
        this.kitchen = kitchen;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            kitchen.prepareOrder();
        }
    }
}

public class RestaurantSystem {
    public static void main(String[] args) {

        Kitchen kitchen = new Kitchen();

        Waiter waiter = new Waiter(kitchen);
        Chef chef = new Chef(kitchen);

        waiter.start();
        chef.start();
    }
}

//OUTPUT:

Waiter placed order: Pizza
Chef is preparing: Pizza
Chef delivered: Pizza

Waiter placed order: Burger
```

```
Chef is preparing: Burger
Chef delivered: Burger

Waiter placed order: Pasta
Chef is preparing: Pasta
Chef delivered: Pasta
```